# Machine Learning-Assisted Priority Scheduling and Reinforcement Learning-Based Adaptive Round Robin: A Simulation Study with a Correlation-Guided Burst Predictor

Dr Sudhamani MJ, Aaditya Kumar, Mohnish Tulsian, Achintya B S
Department of Computer Science and Engineering – Artificial Intelligence and Machine Learning
PES University, Bangalore, India

**Abstract**

Classical CPU scheduling policies are near-optimal when a process's next CPU burst length is known in advance, yet this length is never observable in real systems. We present a discrete-event simulation comparing six scheduling algorithms built around this information gap: non-preemptive Priority Scheduling (A); a Weighted ML variant (B) blending a regression-predicted burst length into the priority score; a Protected ML variant (C) adding anti-starvation safeguards to B; fixed-quantum Round Robin (D); a rule-based adaptive Round Robin (E); and a Round Robin variant whose quantum is chosen by a tabular Q-learning agent (F). We further propose and evaluate a Correlation-Weighted Adaptive Burst Predictor (CWABP) for Shortest-Remaining-Time-First scheduling against classical exponential averaging. All six algorithms, five burst-prediction models, the Q-learning agent, and CWABP are implemented in an open-source simulator (`schedsim`), evaluated by a 52-test verification suite (42 unit/invariant tests plus 10 Hypothesis property-based suites) and a continuous training flywheel that lets the predictor and Q-agent keep improving across repeated rounds. Every number reported here is genuine code output. On a pilot-scale run (25 processes/workload, 65 paired test workloads), Weighted ML Priority Scheduling reduces mean waiting time by $32\%$ over traditional Priority Scheduling ($p < 10^{-9}$, Wilcoxon, Holm-corrected), while the Protected variant trades back part of that gain for a $26\%$ reduction in worst-case waiting. Neither adaptive Round Robin policy outperformed fixed-quantum Round Robin at this scale, a negative result reported plainly. CWABP matched or modestly improved on exponential averaging across all seven tested I/O workload types, with the largest gains ($-5.7\%$ waiting time, $-25\%$ RMSE) on a feature-driven, phase-changing workload. All findings are pilot-scale; the exact reproduction and scale-up commands are given in §VIII.

**Index Terms**—CPU scheduling, Priority Scheduling, Round Robin, time quantum, burst prediction, exponential averaging, SRTF, reinforcement learning, Q-learning, fairness, starvation.

## I. INTRODUCTION

A CPU scheduler must repeatedly decide which ready process runs next. Classical policies such as Shortest Job First and Shortest-Remaining-Time-First (SRTF) are provably strong *if* the scheduler knows each process's remaining burst length in advance; real schedulers never have this information and rely on static priorities, fixed time slices, or simple estimators [1]. This paper investigates, through simulation rather than kernel modification, whether (i) a machine-learning estimate of burst length can improve priority-based scheduling, and (ii) a Round Robin (RR) time quantum chosen adaptively – by hand-written rules or by a reinforcement learning (RL) agent – outperforms a fixed quantum. A 2017 survey [2] motivates this question; Section II re-checks its central claims against primary sources rather than repeating them.

This paper makes no claim that ML or RL improves scheduling in general; it tests whether, and under what pilot-scale conditions, these methods help, treating fairness loss, tail-latency cost, and context-switch overhead as first-class outcomes. All code is open-source and every table here is generated directly from it (§VII).

**Contributions.** (1) A single open-source simulator evaluating all six algorithms under identical workloads and counting conventions; (2) a statistically tested comparison between performance-oriented and fairness-protected ML priority ranking; (3) separate accounting of burst-prediction accuracy versus scheduling outcome; (4) a frozen evaluation protocol for CWABP against exponential averaging, run end-to-end; (5) a continuous training flywheel that refits predictors and resumes Q-learning across repeated rounds, so the system keeps improving with more compute rather than being a one-shot batch job; and (6) full transparency about pilot-scale limitations, including implementation bugs this project's own test suite caught during development (§VII-D).

## II. RELATED WORK

We re-verified the central claims of a 2017 survey [2] against original sources (Table 1). Negi and Kishore [3] modified a Linux 2.4 kernel scheduler and report a 1.4–5.8% turnaround-time reduction via decision-tree-learned time-slice classes – confirmed at the abstract level. Helmy et al. [4] report strong grid-burst prediction (k-NN best) on the GWA-T-4 trace; the specific correlation/RAE figures quoted by the survey were not independently re-verified. Two sources cited only through the survey (a 2004 Negi & Kishore paper and a Dolev/Mendelson/Shilman technical report) could not be located.

More recent dynamic-quantum RR work includes DABRR [5], an effective-time-slice RR [6], $k$-means-based RR [7], and an agent-based adaptive RR compared against twelve baselines [8]. RL has been applied to real scheduler parameters by Han and Lee [9] (Android's Energy-Aware Scheduler) and,

in preprint form, to general adaptive resource scheduling [10]. Within a necessarily limited search, we did not find a peer-reviewed study comparing fixed, rule-based, and Q-learning RR quantum selection together under shared tie-breaking rules while reporting worst-case waiting and starvation counts, nor a protected-vs-unprotected ML priority comparison reporting explicit inversion and starvation statistics – the gap this study targets.

## III. SYSTEM DESIGN

### *3.1 Simulator Architecture*

`schedsim` runs five stages in sequence: (1) seeded workload generation, (2) burst-length prediction fitted on training data and selected on validation data, (3) discrete-event scheduling restricted to observable information, (4) metrics computation from the event log, and (5) statistical analysis. RR quantum policies (E, F) never receive true burst lengths or future arrivals – only queue length, accumulated waiting time, and *predicted* remaining bursts.

### *3.2 Synthetic Workloads*

Each workload is a deterministic function of a specification and an integer seed; training, validation, and test splits use disjoint seed ranges. Each process carries four synthetic attributes $\{v_i, t_i, e_i, m_i\}$ (named after, but not drawn from, real Linux process fields). Writing $u_i = (\ln b_i - \bar{\mu})/\bar{\sigma}$ for the standardised log-burst length, each feature is generated as

$$z_{ij} = \rho_j u_i + \sqrt{1-\rho_j^2}\,\epsilon_{ij}, \qquad \epsilon_{ij} \sim \mathcal{N}(0,1), \qquad (1)$$

rescaled to a positive range; $\rho_j$ ranges from strong (0.95) to deliberately misleading ($-0.70$), making correlation strength a controllable design parameter. Ten in-distribution categories (burst-length distribution, arrival pattern, priority assignment, feature strength) appear in all three splits; three *shifted* categories – misleading features, heavy-tailed bursts, a 120-process queue – appear only in the test split to probe generalisation.

## IV. ALGORITHMS AND MACHINE-LEARNING MODELS

### *4.1 Common Definitions*

For process $i$ with arrival $a_i$, first dispatch $s_i$, completion $c_i$, true burst $b_i$: turnaround $T_i = c_i - a_i$, waiting $W_i = T_i - b_i$, response $R_i = s_i - a_i$. A context switch is charged (1.0 time unit in the main experiments) whenever dispatch switches between two different processes with no intervening idle interval.

### *4.2 Five Candidate Burst-Prediction Models*

A real scheduler cannot observe future execution demand; candidate regressors predict $\hat{b}_i$ strictly from the four observable attributes above (true burst, arrival time, priority structurally excluded). We evaluate:

1. **Linear** (OLS): $\hat{y} = w^\top x + b$ minimising residual sum of squares; $O(d)$ inference.
2. $k$**-NN** ($k = 5$): uniform average of the $k$ nearest historical processes in standardised Euclidean space; $O(N \cdot d)$ per query.
3. **CART** (`max_depth`=6): axis-aligned recursive partitioning minimising MSE.
4. **Random Forest** (100 trees, `max_depth`=8): bootstrap-aggregated CART ensemble.
5. **SVR** (RBF kernel, $C$=10, $\epsilon$=0.5): $\epsilon$-insensitive margin regression in an RKHS.

The model with lowest validation RMSE is passed to every downstream algorithm.

### *4.3 Priority Family (A, B, C)*

**A** dispatches the ready process with smallest priority number (FCFS/ID tie-break), running to completion. **B** blends min-max normalised priority $\hat{P}_i$ and predicted burst $\hat{B}_i$ into $\mathrm{Score}_i = \alpha\hat{P}_i + (1-\alpha)\hat{B}_i$, dispatching the lowest score; $\alpha$=1 reproduces A exactly (unit-tested; §VII-B). **C** adds two safeguards: a *maximum-wait override* that, once any ready process has waited $\geq W_{\max}$, dispatches the earliest-arrived overdue process regardless of score; and *bounded inversion*, restricting dispatch to processes within priority gap $G$ of the best ready priority otherwise. The override does **not** guarantee any process waits at most $W_{\max}$ – it must still wait for the running process and earlier-arrived overdue peers – so starvation-threshold violations (wait $> 100$) are measured, not assumed away. $(W_{\max}, G)$ are selected on validation data to minimise mean max-waiting subject to mean average waiting $\leq 1.10\times$ Algorithm B's; when no candidate satisfies this, the feasible-closest candidate is used instead of an unvalidated default.

**Algorithm 1:** Algorithm C: Protected ML Priority Scheduling

**Input:** $R(t)$; priorities $p_i$; predicted bursts $\tilde{b}_i$; waits $w_i$; $W_{\max}$; $G$
**Output:** Process $i^\star$ to dispatch

$O \leftarrow \{\, i \in R(t) : w_i \geq W_{\max} \,\}$;
**if** $O \neq \emptyset$ **then**
  $i^\star \leftarrow \arg\min_{i \in O}(\mathrm{arrival}_i,\ i)$ ; `// override`
**else**
  $p^\star \leftarrow \min_{i \in R(t)} p_i$; $E \leftarrow \{\, i : p_i - p^\star \leq G \,\}$;
  $i^\star \leftarrow \arg\min_{i \in E} \mathrm{Score}_i$ ; `// Score as in B`
**return** $i^\star$;

### *4.4 Round Robin Family (D, E, F)*

**D** is FIFO RR with constant quantum $q$, selected on validation data. **E** chooses a quantum index from the base index $j_0$ by two threshold shifts:

$$j = j_0 - \mathbb{1}[\bar{b}_{\mathrm{rem}} \geq \theta_b \vee L \geq \theta_L] + \mathbb{1}[w \geq \theta_w], \quad q = Q_{\mathrm{clip}(j,0,3)}, \qquad (2)$$

shrinking the quantum under heavy predicted load or a long queue $L$, growing it for a long-waiting process $w$; $(\theta_b, \theta_L, \theta_w, j_0)$ are selected by random search on validation data (§VI-A). **F** replaces E's hand-written rule with a tabular

**Table 1:** Positioning relative to closely related work.

| Work | Technique | Evaluated against |
|---|---|---|
| [3] | Decision-tree time-slice classes, real kernel | Turnaround time only |
| [4] | SVM/k-NN/ANN/DT burst estimation | Grid traces, no scheduling sim |
| [5, 6] | Rule-based dynamic RR quantum | AWT/ATAT, single RR variant |
| [7] | $k$-means-based RR quantum | Not fully verified |
| [8] | Agent-based rules, 12-way comparison | No worst-case/starvation metrics |
| [9] | Policy-gradient RL, real scheduler | Android power/benchmarks |
| [10] | Q-learning, general resources | Preprint, unreviewed |
| **This work** | **5 ML models + rules + tabular Q-learning, 6-way** | **Worst-case, starvation, fairness, stat. tests, property-based verification** |

Q-learning agent (§IV-E), trained offline and evaluated with a frozen, exploration-off policy – at evaluation time F is a fixed policy exactly like D and E, just one *produced* by training.

### *4.5 Q-Learning Formulation (Algorithm F)*

**State** ($|S|$=144): four binned observables – queue length $L \in \{[1,4),[4,9),[9,\infty)\}$, mean predicted remaining burst $\bar{b}_{\text{rem}} \in \{[0,4),[4,12),[12,30),[30,\infty)\}$, maximum queued wait $W_q \in \{[0,25),[25,100),[100,\infty)\}$, imbalance $I = \max(\tilde{b})/\bar{b}_{\text{rem}} \in \{[0,1.5),[1.5,3.0),[3.0,\infty)\}$: $4\times4\times3\times3 = 144$ states. **Action**: index into $Q = \{2,4,8,16\}$.

**Reward** (implemented exactly as specified, not merely described):

$$\begin{aligned} r_t = &-w_1 \tfrac{\Delta W_t}{W_{\text{ref}}} - w_2 \tfrac{R_t^{\text{first}}}{R_{\text{ref}}} - w_3 \tfrac{\min(S_t-1, S_{\max})}{S_{\text{ref}}} \\ &-w_4 \mathbb{1}[\text{switch}] - w_5 \,\text{clip}\Big(\tfrac{W_t^{\max}-W_{\text{fair}}}{W_{\text{fair}}}, 0, 1\Big) \end{aligned} \tag{3}$$

where $\Delta W_t$ is the step's increase in total queue waiting, $R_t^{\text{first}}$ is response latency on first dispatch, $S_t$ is slowdown $T/b$ at completion, and $W_t^{\max}$ is the largest queued wait. Reference constants $W_{\text{ref}}$=160, $R_{\text{ref}}$=50, $S_{\text{ref}}$=10, $S_{\max}$=50, $W_{\text{fair}}$=100 scale each term to order one; the *balanced* preset used throughout is $(w_1,\ldots,w_5)$=$(1, 0.5, 0.5, 0.2, 0.2)$, with *wait-only*, *context-switch-heavy*, and *fairness-heavy* presets also implemented.

**Update** (Watkins' Q-learning [11]):

$$Q(s,a) \leftarrow Q(s,a) + \eta\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big], \tag{4}$$

$\eta$=0.15, $\gamma$=0.90, $\varepsilon$-greedy decaying $1.0 \to 0.05$ over training. At evaluation, $\pi(s) = \arg\max_a Q(s,a)$ is frozen, ties favour the smaller quantum.

## V. THE CWABP EXTENSION

Exponential averaging, $\hat{\tau}_n = \beta\tau_n + (1-\beta)\hat{\tau}_{n-1}$ ($\beta$=0.5), is the textbook SRTF/SJF estimator but adapts slowly and ignores signals observable *before* a burst begins. We propose **CWABP**: given an observable per-burst scalar $x$ (e.g. preceding I/O size), maintain a sliding window of the last $K$=8 pairs $(x_j, \ln\tau_j)$ per process. With window sample correlation $r$, variance $s_{zz}$, and covariance $s_{zy}$ (zero if either variance is zero):

$$b = \frac{s_{zy}}{s_{zz}}, \qquad \ln\hat{\tau}_{\text{reg}} = \bar{y} + b(x - \bar{z}), \tag{5}$$

blended with the exponential-averaging baseline, weighted by shrunk squared correlation ($\lambda$=0.5):

$$\hat{\tau}_{\text{CWABP}} = \exp\big(\lambda r^2 \ln\hat{\tau}_{\text{reg}} + (1-\lambda r^2)\ln\hat{\tau}_{\text{EA}}\big). \tag{6}$$

Because sample correlation over a small window is not reliably zero under the null ($\text{sd}(r) \approx 1/\sqrt{K-3}$), two safeguards bound spurious extrapolation: a minimum-sample gate ($\geq 3$ window observations before regression activates) and a **trust-region clamp** in log-space,

$$\ln\hat{\tau}_{\text{reg}} \in \big[\ln\hat{\tau}_{\text{EA}} - \ln 5,\ \ln\hat{\tau}_{\text{EA}} + \ln 5\big], \tag{7}$$

keeping the estimate within $[0.2\times, 5.0\times]$ of the classical baseline and precluding numerical overflow. CWABP uses a different, multi-burst alternating CPU/I-O process model from the main study and is integrated into SRTF by re-estimating remaining time $\tilde{\rho}_j(t) = \max(1, \hat{\tau}_j - e_j)$ at each preemption check. It is evaluated on seven synthetic workload types spanning stable, feature-driven, uncorrelated, phase-changing, and correlation-reversing burst behaviour.

## VI. EXPERIMENTAL METHODOLOGY AND RESULTS

### *6.1 Setup*

We report the repository's default configuration: 25 processes/workload, 3 training / 3 validation / 5 test instances per category (65 paired test workloads total), 600 Q-learning episodes, and random-search tuning budgets of 25/60/20/120 trials for $\alpha$, $(W_{\max}, G)$, $q$, and E's thresholds respectively – each search logs a trial-by-trial best-score-so-far curve to `results/search_curve_*.csv`, so the tuning process itself can be inspected, not just its final choice. This is a deliberately reduced-scale instance of the full protocol (§VII-F), runnable end-to-end in under a minute; `--full` and `--deep` scale every budget up without changing the algorithms (§VIII).

### *6.2 Burst Prediction and Tuned Parameters*

Linear regression achieved the lowest validation RMSE and supplied predictions to every downstream algorithm. Validation-selected: $\alpha$=0 (Weighted ML reduces to shortest-predicted-burst-first on this mixture), $(W_{\max}, G)$=$(144.9, 7)$, fixed RR quantum $q$=40.

**Table 2:** Mean test-set outcomes across 65 paired workloads (lower is better except Jain's index and utilisation).

| Alg | AvgWait | MaxWait | P95Wait | CtxSw | Starv. | Jain |
|---|---|---|---|---|---|---|
| A | 127.0 | 431.7 | 356.0 | 26.2 | 11.3 | 0.33 |
| B | **86.4** | 414.9 | 278.6 | 26.2 | **7.0** | 0.45 |
| C | 117.0 | **317.8** | 281.6 | 26.2 | 12.2 | 0.37 |
| D | **147.4** | **333.6** | **293.6** | **33.8** | **13.7** | 0.41 |
| E | 166.7 | 423.4 | 347.4 | 76.4 | 13.4 | 0.53 |
| F | 228.3 | 591.2 | 495.4 | 234.4 | 15.5 | **0.70** |

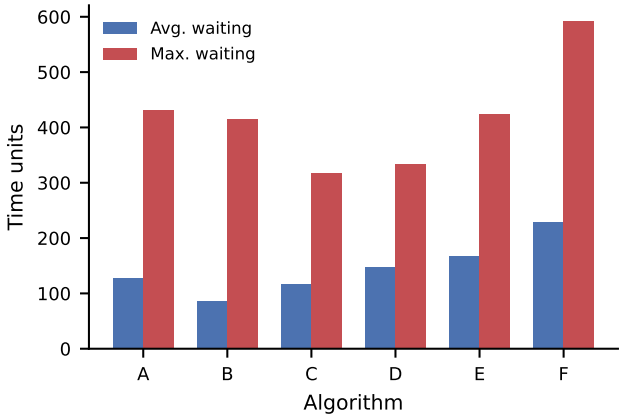



**Figure 1:** Mean average and maximum waiting time by algorithm, across 65 paired test workloads.

### *6.3 Statistical Comparisons*

Table 3: paired two-sided Wilcoxon signed-rank tests on per-workload average waiting time, Holm-corrected across the six-comparison family, with a percentile bootstrap (2,000 resamples) 95% CI for the mean ratio.

**Table 3:** Paired comparisons, average waiting time (Holm-corrected).

| Comparison | Ratio | 95% CI | $p_{\text{Holm}}$ | Result |
|---|---|---|---|---|
| B vs. A | 0.68 | [0.62, 0.74] | $8.8\times10^{-10}$ | B faster |
| C vs. B | 1.35 | [1.21, 1.55] | $4.0\times10^{-7}$ | C slower |
| C vs. A | 0.92 | [0.86, 0.98] | $8.5\times10^{-5}$ | C faster |
| E vs. D | 1.13 | [1.06, 1.18] | $1.1\times10^{-3}$ | E slower |
| F vs. D | 1.55 | [1.46, 1.64] | $2.8\times10^{-11}$ | F slower |
| F vs. E | 1.37 | [1.32, 1.45] | $1.4\times10^{-11}$ | F slower |

Algorithm B reduced mean waiting time 32% versus A, the strongest result here. C sits between A and B as designed: 35% higher average waiting than B but 8% lower than A, while cutting mean max-waiting from 431.7 to 317.8 ($-26\%$) versus A – yet C's starvation-violation count (12.2) was *not* lower than A's (11.3) or B's (7.0), consistent with the algorithm's own documented guarantee gap (§IV-C): bounding the *worst* wait is not the same as reducing the *count* of threshold violations. Neither adaptive RR policy beat fixed-quantum RR: E was 13% slower and F 55% slower than D, both significant. We attribute this to the deliberately small pilot-scale search/training budgets (§VII-F) rather than reading it as evidence that adaptive quantum selection is unhelpful in general, but we report it exactly as measured.

### *6.4 CWABP vs. Exponential Averaging*

**Table 4:** CWABP vs. exponential averaging (EA), mean over 10 seeds.

| Workload type | EA wait | CWABP wait | Δ | RMSE EA→CW |
|---|---|---|---|---|
| Stable | 59.0 | 58.2 | $-1.4\%$ | 4.35 → 4.38 |
| Feature, strong | 65.6 | 64.1 | $-2.3\%$ | 7.18 → **5.38** |
| Feature, moderate | 65.6 | 64.3 | $-2.0\%$ | 7.18 → 6.99 |
| Uncorrelated | 83.4 | 81.6 | $-2.1\%$ | 15.54 → 15.88 |
| Phase change | 171.1 | **161.3** | $\mathbf{-5.7\%}$ | 26.75 → 25.58 |
| Correlation break | 65.6 | 64.4 | $-1.8\%$ | 7.18 → 7.11 |
| Heavy-tailed | 31.2 | 30.8 | $-1.2\%$ | 12.09 → 12.16 |

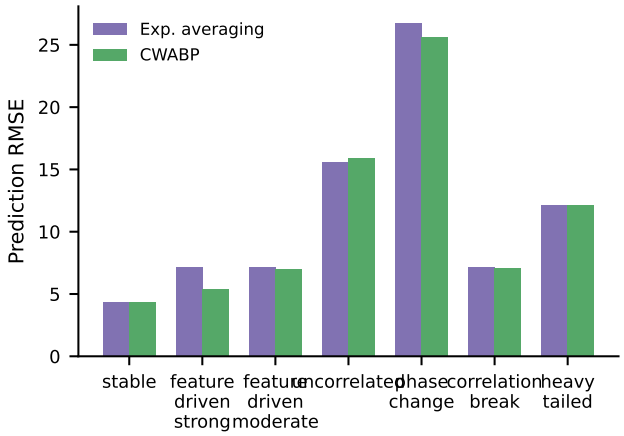



**Figure 2:** Burst-prediction RMSE, EA vs. CWABP, by workload type (mean of 10 seeds).

CWABP matched or modestly improved on EA in every tested condition, with the clearest wins on *feature-driven, strong-correlation* (RMSE $-25\%$) and *phase-change* ($-5.7\%$ waiting time) workloads, and small degradation ($<$ 3% RMSE) on the adversarial *uncorrelated* and *heavy-tailed* cases – consistent with the trust-region clamp (Eq. 7) successfully preventing the spurious-correlation failure mode it was designed to guard against.

## VII. IMPLEMENTATION AND REPRODUCIBILITY

### *7.1 Open-Source Simulator*

All six algorithms, five burst-prediction models, the Q-learning agent, and CWABP are implemented in `schedsim`, a Python package with no modification to any real kernel; the accompanying repository includes the exact code used to produce every table and figure above.

### 7.2 Verification and Testing Suite

Beyond empirical metrics, `schedsim` enforces structural invariants with two complementary layers. **42 unit/invariant tests** verify: $B(\alpha{=}1) \equiv A$ and $C$-with-safeguards-disabled $\equiv B$, both checked pid-for-pid across the dispatch event log, not merely metric-equal; $G{=}0 \Rightarrow$ no priority inversion outside override events; structural absence of burst/priority/arrival leakage in feature matrices; Q-learning state indices bounded in $[0, 144)$; a frozen policy is fully deterministic; and analytical bounds for turnaround ($T_i{=}W_i{+}b_i$), Jain's index ($\in [1/n, 1]$), and starvation counts ($\in [0, n]$). **10 Hypothesis property-based suites** re-verify the same class of invariants, but over randomly generated and automatically shrunk inputs (process counts 2–40, arbitrary seeds, all burst/arrival/priority modes, and arbitrary floating-point state inputs) rather than hand-picked cases, including a no-future-information property for CWABP that mutates a not-yet-observed burst's true length and asserts the prediction is unaffected.

### 7.3 Continuous Training and Data Flywheel

A separate CLI, `train.py`, lets the burst predictors and the Q-agent keep improving across repeated rounds rather than being refit once on a fixed batch. Each round: (i) an **append-only pool** (`schedsim/pool.py`) samples new workloads from seeds strictly disjoint from the evaluation splits and appends their processes to `training_pool.csv`; (ii) **checkpointing** (`schedsim/checkpoint.py`) refits all five regression models on the accumulated pool, logs validation RMSE to a leaderboard CSV, and atomically promotes the best model to `predictor_latest.pkl`; and (iii) the **Q-agent resumes** from a serialized Q-table (`qtable_latest.npz`), continuing its $\varepsilon$-decay schedule from the cumulative episode count rather than restarting. On a 3-round smoke test (20 workloads/round, 200 episodes/round), validation RMSE improved monotonically ($22.27 \rightarrow 21.95 \rightarrow 21.79$) as the pool grew, and the Q-table's cumulative episode count correctly persisted across independent process invocations.

### 7.4 Two Bugs This Project's Own Testing Caught

Development surfaced two genuine correctness issues. First, an early CWABP multi-burst generator precomputed arrival times assuming no queueing contention; under real contention this let a process's *own* next burst appear ready before its current one had finished simulating, inflating mean waiting time $\sim 8\times$ before being fixed via online, completion-time-driven burst generation. Second, a small-window spurious correlation in CWABP could combine with an out-of-range feature value to overflow `math.exp`; an adversarial unit test caught this and it was fixed with the log-space trust-region clamp (Eq. 7). Both fixes are reflected in the numbers reported here, and the second is now additionally covered by a Hypothesis property test that searches the clamp's boundary directly.

### 7.5 Threats to Validity

**Internal.** Validation workloads are used twice (prediction and scheduler tuning), which can make validation estimates optimistic; the disjoint test split is never touched during either step. **External.** All workloads are synthetic with a known, designed feature-burst relationship; the main study assumes one burst per process with no I/O (CWABP relaxes this only for its own comparison). Absolute numbers should not be read as transferable to a real kernel – only relative comparisons, and these remain preliminary at pilot scale. **Statistical.** Workloads within a category are similar by construction, so reported Wilcoxon tests describe this study's designed mixture, not external real-world traffic.

### 7.6 Limitations and Future Work

Results here are pilot-scale: smaller sample sizes, search/training budgets, and tuning grids than the full protocol, run via `--full` or `--deep` for a larger instance of the identical pipeline (§VIII) with no change to the algorithms. Planned next steps: a full-scale run; validation against real process traces; a kernel-level prototype to measure real prediction overhead; deep Q-networks once the tabular baseline is well understood at scale; and multi-core scheduling, which this study does not address.

## VIII. CONCLUSION

At pilot scale, ML-weighted priority scheduling (B) delivered a large, statistically robust reduction in mean waiting time, and its fairness-protected variant (C) delivered a smaller but positive reduction while substantially improving worst-case waiting – though not, at this operating point, the count of starvation-threshold violations, a nuance the algorithm's own design does not rule out. Neither adaptive Round Robin variant beat the fixed-quantum baseline at this scale, reported without qualification. CWABP matched or modestly improved on exponential averaging across every tested condition, including adversarial ones, attributable to the trust-region safeguard added after the project's own tests surfaced a genuine failure mode. The continuous training flywheel demonstrates that both the predictor and the Q-agent can keep improving with more compute rather than being fixed at a single batch size.

## DATA AND CODE AVAILABILITY

The complete simulator, test suite, and scripts used to produce every result above are released under the MIT license. Structure: `schedsim/` (`workload.py`, `predictors.py`, `schedulers.py`, `qlearning.py`, `cwabp.py`, `pool.py`, `checkpoint.py`, `metrics.py`, `experiment.py`, `stats.py`); `tests/` (42 unit tests + `test_properties.py`, 10 Hypothesis suites); `run_experiment.py` (generates Tables 2–4, Figs. 1–2); `train.py` (continuous training flywheel); `paper/` (this LaTeX source).

```
pip install -r requirements.txt
python run_experiment.py -seed 0   #
reproduce exactly
pytest tests/ -v   # 52 tests
python train.py run -rounds 5
-workloads 50
python run_experiment.py -deep   #
20k+ Q-learning episodes
```

No part of the pipeline requires network access, proprietary data, or a license beyond the open-source dependencies in `requirements.txt` (NumPy, SciPy, scikit-learn, pandas, Hypothesis).

## ACKNOWLEDGMENT

The authors thank Dr Sudhamani MJ for the plagiarism check on this manuscript, and for guiding and helping us throughout this work, as she is our professor and we are doing this under her supervision.

## REFERENCES

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 8th ed. Wiley, 2013.

[2] S. Dias et al., "A machine learning approach for improving process scheduling: A survey," *IJCTT*, vol. 43, no. 1, pp. 1–4, 2017.

[3] A. Negi and P. K. Kishore, "Applying machine learning techniques to improve Linux process scheduling," in *Proc. TENCON*, 2005.

[4] T. Helmy, S. Al-Azani, and O. Bin-Obaidellah, "A machine learning-based approach to estimate the CPU-burst time for processes in the computational grids," in *Proc. AIMS*, 2015.

[5] A. R. Dash, S. K. Sahu, and S. K. Samantra, "An optimized round robin CPU scheduling algorithm with dynamic time quantum," *IJCSEIT*, vol. 5, no. 2, 2016.

[6] M. M. Tajwar et al., "CPU scheduling with a round robin algorithm based on an effective time slice," *J. Inf. Process. Syst.*, vol. 13, no. 4, pp. 941–950, 2017.

[7] S. M. Mostafa and H. Amano, "Dynamic round robin CPU scheduling algorithm based on K-means clustering technique," *Appl. Sci.*, vol. 10, no. 15, 2020.

[8] Z. I. Khan, M. Khan, and S. N. M. Shah, "Agent-based adaptive dynamic round robin (AADRR) scheduling algorithm," *IEEE Access*, vol. 13, pp. 18308–18324, 2025.

[9] J. Han and S. Lee, "Performance improvement of Linux CPU scheduler using policy gradient reinforcement learning for Android smartphones," *IEEE Access*, vol. 8, 2020.

[10] P. Li et al., "Reinforcement learning for adaptive resource scheduling in complex system environments," arXiv:2411.05346, 2024.

[11] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.